ELASTICSEARCH WORKSHOP

Category 8: Nested Fields and Queries

EXERCISES

Scenario

You are a data engineer for a fictional intelligence unit called UNIT-7.
Your unit maintains a profile for each operative. Each operative carries multiple pieces of equipment.
Equipment is stored as an array of objects inside each operative document.
You will use nested fields so you can query each equipment item as an independent object.
All names, codes, and equipment types are completely fictional.

Key Concept: The Problem with Object Arrays

In Elasticsearch, when you store an array of objects without nested type, the fields are flattened.

Example: one operative has two equipment items:

```
# Item 1: { 'item': 'Radio',  'condition': 'good' }
# Item 2: { 'item': 'Knife',  'condition': 'worn' }
#
# Without nested type, Elasticsearch stores it as:
#   equipment.item:      ['Radio', 'Knife']
#   equipment.condition: ['good',  'worn']
#
# A query for item='Radio' AND condition='worn' would MATCH this document.
# This is WRONG. No single equipment item has both Radio and worn.
# The connection between item and condition is lost.
```

The solution is to map equipment as type 'nested'. This preserves the connection between fields inside each object.

With nested type, each equipment item is stored as a hidden separate document.

A nested query searches inside each item independently.

Rule: use nested type whenever you have an array of objects and you need to
query multiple fields on the same object at the same time.

SQL analogy: nested is like a one-to-many child table joined to the parent.
MongoDB analogy: similar to querying embedded documents with $elemMatch.

Dataset: Operative Equipment Profiles

Index name: 'operatives' | 6 operative documents

Each operative document has these top-level fields:

Field	Type	Notes
operative_id	keyword	Unique ID like 'OP-001'.
call_sign	keyword	Operative code name.
unit	keyword	Assigned unit: ALPHA, BRAVO, or CHARLIE.
rank	keyword	Field rank.
active	boolean	Whether the operative is currently active.
equipment	nested	Array of equipment objects. See below.

Each item in the equipment array has these fields:

Field	Type	Notes
item	keyword	Name of the equipment item.
category	keyword	Category: weapon, comms, medical, navigation.
condition	keyword	Condition: new, good, worn, damaged.
issued_year	integer	Year the item was issued.

The 6 operative records:

```
OP-001 | call_sign: Ghost  | unit: ALPHA   | rank: Captain  | active: true
  equipment:
    { item: Rifle-X4,    category: weapon,     condition: good,    issued_year: 2022 }
    { item: Radio-T7,    category: comms,      condition: new,     issued_year: 2024 }
    { item: MedKit-A,    category: medical,    condition: good,    issued_year: 2023 }

OP-002 | call_sign: Falcon | unit: BRAVO   | rank: Lieutenant | active: true
  equipment:
    { item: Pistol-P3,   category: weapon,     condition: worn,    issued_year: 2020 }
    { item: Radio-T7,    category: comms,      condition: good,    issued_year: 2023 }
    { item: GPS-Nav2,    category: navigation, condition: new,     issued_year: 2024 }
```

```
OP-003 | call_sign: Viper  | unit: ALPHA   | rank: Sergeant   | active: false
  equipment:
    { item: Rifle-X4,    category: weapon,     condition: damaged, issued_year: 2019 }
    { item: MedKit-A,    category: medical,    condition: worn,    issued_year: 2021 }

OP-004 | call_sign: Shadow | unit: CHARLIE | rank: Captain    | active: true
  equipment:
    { item: Pistol-P3,   category: weapon,     condition: good,    issued_year: 2023 }
    { item: Radio-T7,    category: comms,      condition: worn,    issued_year: 2021 }
    { item: GPS-Nav2,    category: navigation, condition: good,    issued_year: 2022 }
    { item: MedKit-A,    category: medical,    condition: new,     issued_year: 2024 }

OP-005 | call_sign: Hawk   | unit: BRAVO   | rank: Lieutenant | active: true
  equipment:
    { item: Rifle-X4,    category: weapon,     condition: new,     issued_year: 2024 }
    { item: GPS-Nav2,    category: navigation, condition: worn,    issued_year: 2020 }

OP-006 | call_sign: Dagger | unit: CHARLIE | rank: Sergeant   | active: false
  equipment:
    { item: Pistol-P3,   category: weapon,     condition: damaged, issued_year: 2018 }
    { item: Radio-T7,    category: comms,      condition: damaged, issued_year: 2019 }
    { item: MedKit-A,    category: medical,    condition: worn,    issued_year: 2022 }
```

Exercise 1 [Step-by-step] (required)

Task: Create the 'operatives' index with nested mapping for equipment. Index all 6 operative documents.

Step 1 — Create the index with nested mapping

```
from elasticsearch import Elasticsearch

es = Elasticsearch('http://localhost:9200')

if es.indices.exists(index='operatives'):
    es.indices.delete(index='operatives')

# The equipment field is type 'nested'.
# Inside nested, list all the sub-fields and their types.
# Without 'nested', Elasticsearch would flatten the array and lose field
connections.
mapping = {
    'mappings': { 'properties': {
        'operative_id': { 'type': 'keyword' },
        'call_sign':    { 'type': 'keyword' },
        'unit':         { 'type': 'keyword' },
        'rank':         { 'type': 'keyword' },
```

```
        'active':       { 'type': 'boolean' },
        'equipment': {
            'type': 'nested',
            'properties': {
                'item':        { 'type': 'keyword' },
                'category':    { 'type': 'keyword' },
                'condition':   { 'type': 'keyword' },
                'issued_year': { 'type': 'integer' }
            }
        }
    }}
}
es.indices.create(index='operatives', body=mapping)
print('Index created.')
```

Step 2 — Index all 6 operative documents

```
from elasticsearch.helpers import bulk

operatives = [
  {'_index':'operatives','_id':'OP-001','_source':{

'operative_id':'OP-001','call_sign':'Ghost','unit':'ALPHA','rank':'Captain','active
':True,
    'equipment':[
      {'item':'Rifle-X4','category':'weapon',    'condition':'good',
'issued_year':2022},
      {'item':'Radio-T7','category':'comms',     'condition':'new',
'issued_year':2024},
      {'item':'MedKit-A','category':'medical',   'condition':'good',
'issued_year':2023}
    ]}},
  {'_index':'operatives','_id':'OP-002','_source':{

'operative_id':'OP-002','call_sign':'Falcon','unit':'BRAVO','rank':'Lieutenant','ac
tive':True,
    'equipment':[
      {'item':'Pistol-P3','category':'weapon',     'condition':'worn',
'issued_year':2020},
      {'item':'Radio-T7', 'category':'comms',      'condition':'good',
'issued_year':2023},
      {'item':'GPS-Nav2', 'category':'navigation', 'condition':'new',
'issued_year':2024}
    ]}},
  {'_index':'operatives','_id':'OP-003','_source':{

'operative_id':'OP-003','call_sign':'Viper','unit':'ALPHA','rank':'Sergeant','activ
e':False,
    'equipment':[
      {'item':'Rifle-X4','category':'weapon',
'condition':'damaged','issued_year':2019},
      {'item':'MedKit-A','category':'medical', 'condition':'worn',
'issued_year':2021}
    ]}},
  {'_index':'operatives','_id':'OP-004','_source':{

'operative_id':'OP-004','call_sign':'Shadow','unit':'CHARLIE','rank':'Captain','act
ive':True,
```

```
    'equipment':[
      {'item':'Pistol-P3','category':'weapon',
'condition':'good','issued_year':2023},
      {'item':'Radio-T7', 'category':'comms',
'condition':'worn','issued_year':2021},
      {'item':'GPS-Nav2', 'category':'navigation',
'condition':'good','issued_year':2022},
      {'item':'MedKit-A', 'category':'medical',    'condition':'new',
'issued_year':2024}
    ]}},
  {'_index':'operatives','_id':'OP-005','_source':{

'operative_id':'OP-005','call_sign':'Hawk','unit':'BRAVO','rank':'Lieutenant','acti
ve':True,
    'equipment':[
      {'item':'Rifle-X4','category':'weapon',     'condition':'new',
'issued_year':2024},
      {'item':'GPS-Nav2','category':'navigation',
'condition':'worn','issued_year':2020}
    ]}},
  {'_index':'operatives','_id':'OP-006','_source':{

'operative_id':'OP-006','call_sign':'Dagger','unit':'CHARLIE','rank':'Sergeant','ac
tive':False,
    'equipment':[
      {'item':'Pistol-P3','category':'weapon',
'condition':'damaged','issued_year':2018},
      {'item':'Radio-T7', 'category':'comms',
'condition':'damaged','issued_year':2019},
      {'item':'MedKit-A', 'category':'medical', 'condition':'worn',
'issued_year':2022}
    ]}}
]
success, errors = bulk(es, operatives)
print('Indexed:', success, '| Errors:', errors)
```

Exercise 2 [Step-by-step] (required)

Task: Find all operatives who have at least one weapon in damaged condition.

A nested query searches inside nested objects. Both field conditions must match the SAME object. Use path 'equipment' and match: category = weapon AND condition = damaged.

Step 1 — Write the nested query

```
from elasticsearch import Elasticsearch

es = Elasticsearch('http://localhost:9200')

# nested query: searches inside each equipment item independently.
# 'path': the name of the nested field.
# 'query': conditions that must be true for the SAME equipment item.
```

```
# Both category=weapon AND condition=damaged must be in the same item.
query = {
    'query': {
        'nested': {
            'path': 'equipment',
            'query': {
                'bool': {
                    'filter': [
                        { 'term': { 'equipment.category':  'weapon'  } },
                        { 'term': { 'equipment.condition': 'damaged' } }
                    ]
                }
            }
        }
    }
}
```

Step 2 — Run and print results

```
response = es.search(index='operatives', body=query)
hits = response['hits']['hits']

print('Operatives with a damaged weapon:', len(hits))
for hit in hits:
    src = hit['_source']
    print(src['operative_id'], '-', src['call_sign'])
    # Print which equipment items are damaged weapons
    for item in src['equipment']:
        if item['category'] == 'weapon' and item['condition'] == 'damaged':
            print('   ->', item['item'], '(damaged weapon)')

# Expected output:
# Operatives with a damaged weapon: 2
# OP-003 - Viper  -> Rifle-X4 (damaged weapon)
# OP-006 - Dagger -> Pistol-P3 (damaged weapon)
```

Command reference:

```
 'nested': {
    'path':  'field_name',
    'query': { ... conditions on sub-fields ... }
 }
```

Field names inside a nested query must be prefixed with the path.
Example: 'equipment.category', not just 'category'.

Without nested type and nested query:
A query for category=weapon AND condition=damaged would also match OP-001
because OP-001 has a weapon (Rifle-X4, good) AND a damaged... wait, it has no damaged item.
The flattening problem becomes clear with more data.

```
MongoDB analogy: db.operatives.find({ equipment: { $elemMatch: { category: 'weapon', condition:
'damaged' } } })
```

Exercise 3

Task: Find all operatives who have at least one equipment item issued before 2021.

Use a nested query with a range condition on equipment.issued_year.
issued_year < 2021 means: lt 2021.

Print the operative_id, call_sign, and which item(s) are old.

Expected result: OP-002 (Pistol-P3, 2020), OP-003 (Rifle-X4, 2019), OP-005 (GPS-Nav2, 2020), OP-006 (Pistol-P3, 2018 and Radio-T7, 2019).

```
Command reference:
  'nested': {
      'path': 'equipment',
      'query': { 'range': { 'equipment.issued_year': { 'lt': 2021 } } }
  }

  Range queries inside nested work the same way as at the top level.
  Always prefix the field name with the path: equipment.issued_year
```

Exercise 4 (required)

Task: Find active operatives who have at least one comms item in good condition.

This requires two conditions:
- Top-level: active = true (use a bool filter)
- Nested: equipment.category = comms AND equipment.condition = good

Combine them using a bool query. The nested query goes inside the must or filter list.

Expected result: OP-001 (Ghost, Radio-T7 good), OP-002 (Falcon, Radio-T7 good).

```
Command reference:
```

```
'bool': {
    'filter': [
        { 'term': { 'active': True } },
        { 'nested': {
            'path': 'equipment',
            'query': { 'bool': { 'filter': [
                { 'term': { 'equipment.category':  'comms' } },
                { 'term': { 'equipment.condition': 'good'  } }
            ]}}
        }}
    ]
}

A nested query can be placed anywhere a normal query clause can go.
Here it is one item in the bool filter list.
```

Exercise 5

Task: Find all operatives who have a GPS-Nav2 item.

Use a nested query with a single term condition: equipment.item = 'GPS-Nav2'.

Expected result: OP-002 (Falcon), OP-004 (Shadow), OP-005 (Hawk).

```
Command reference:
 'nested': {
    'path': 'equipment',
    'query': { 'term': { 'equipment.item': 'GPS-Nav2' } }
 }

 You can use a single query clause inside nested, not only bool.
 A simple term query is valid inside nested.
```

Exercise 6

Task: Find operatives who have a medical item issued in 2023 or later. Show which specific items matched.

Use a nested query with inner_hits. inner_hits returns the specific nested objects that matched the query, not just the parent document.

Expected result: OP-001 (MedKit-A, 2023), OP-004 (MedKit-A, 2024).

```
Command reference:
 'nested': {
     'path': 'equipment',
     'query': { 'bool': { 'filter': [
         { 'term':  { 'equipment.category':    'medical' } },
         { 'range': { 'equipment.issued_year': { 'gte': 2023 } } }
     ]}},
     'inner_hits': {}
 }

 'inner_hits': {}   Include the matching nested objects in the response.

 Access inner_hits:
 hit['inner_hits']['equipment']['hits']['hits']
 Each inner hit has '_source' with the equipment item fields.

 inner_hits is very useful when you need to know WHICH item matched, not just that one did.
```

Exercise 7

Task: Count how many equipment items exist per category across all operatives.

Aggregations on nested fields require a special nested aggregation wrapper.
The nested aggregation enters the nested context. Then you put a terms agg inside it.

Expected result: weapon=6, comms=5, medical=5, navigation=3. (Total 19 items across all operatives.)

```
Command reference:
 'aggs': {
     'equipment_items': {
         'nested': { 'path': 'equipment' },
         'aggs': {
             'by_category': {
                 'terms': { 'field': 'equipment.category', 'size': 10 }
             }
         }
     }
 }

 The outer 'nested' agg navigates into the nested documents.
```

```
The inner 'terms' agg runs on those nested documents.
Without the nested wrapper, a terms agg on equipment.category would return nothing.

Access: response['aggregations']['equipment_items']['by_category']['buckets']
```

Exercise 8

Task: Count how many damaged or worn equipment items there are per category.

Use a nested aggregation. Inside it, add a filter aggregation to keep only damaged or worn items. Then add a terms aggregation on category.

Expected result: weapon=3 (Pistol-P3 worn, Rifle-X4 damaged, Pistol-P3 damaged), medical=3 (worn + worn + worn), comms=2 (worn + damaged + damaged), navigation=2 (GPS-Nav2 worn + GPS-Nav2 worn).

```
Command reference:
 'aggs': {
    'all_equipment': {
        'nested': { 'path': 'equipment' },
        'aggs': {
            'degraded_items': {
                'filter': { 'terms': { 'equipment.condition': ['damaged','worn'] } },
                'aggs': {
                    'by_category': { 'terms': { 'field': 'equipment.category', 'size': 10 } }
                }
            }
        }
    }
 }

 Access buckets:
 response['aggregations']['all_equipment']['degraded_items']['by_category']['buckets']
```

Exercise 9

Task: For each equipment category, count how many DISTINCT operatives (parent documents) have an item in that category.

Use nested aggregation > terms on category > reverse_nested aggregation.

reverse_nested exits the nested context and counts the parent documents.

Expected result: weapon=6 operatives (all have a weapon), comms=4 operatives, medical=4 operatives, navigation=3 operatives.

```
Command reference:
 'by_category': {
    'terms': { 'field': 'equipment.category', 'size': 10 },
    'aggs': {
       'operative_count': {
          'reverse_nested': {}
       }
    }
 }

 reverse_nested: {} exits the nested context. The count is parent documents, not items.
 bucket['operative_count']['doc_count']  = number of distinct operatives.
```

Exercise 10

Task: Find all active operatives who have at least one weapon in new or good condition AND at least one comms item. Show which matched equipment items using inner_hits.

Use a bool query:
- filter: active = true
- filter: nested query for weapon AND (condition = new OR condition = good)
- filter: nested query for comms (any condition)

Add inner_hits to each nested query so you can see which items matched.

Expected result: OP-001 (Ghost), OP-004 (Shadow).

```
Command reference:
 'bool': {
    'filter': [
       { 'term': { 'active': True } },
       { 'nested': {
          'path': 'equipment',
          'query': { 'bool': { 'filter': [
             { 'term': { 'equipment.category': 'weapon' } },
             { 'terms': { 'equipment.condition': ['new','good'] } }
          ]}},
```

```
            'inner_hits': { 'name': 'matched_weapons' }
        }},
        { 'nested': {
            'path': 'equipment',
            'query': { 'term': { 'equipment.category': 'comms' } },
            'inner_hits': { 'name': 'matched_comms' }
        }}
    ]
}

When using multiple inner_hits, give each a unique 'name'.
Access: hit['inner_hits']['matched_weapons']['hits']['hits']
        hit['inner_hits']['matched_comms']['hits']['hits']
```

End of Category 8 Exercises